The Ministry of Education and Science of Ukraine
National Technical University of Ukraine
“Kyiv Polytechnic Institute named after Igor Sikorsky”

Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy

Methods of synthesis of virtual reality

Calculation and graphics work

(Spatial audio)

Variant 17

Date “04” May 2025

Performed by: 1st year master’s student
group TR-43mp
Arina Kholodnytska

Evaluation “_____”
Date “___” _____ 20___

Checked:
PhD in Engineering, Senior lecturer,
Anatoliy Demchyshyn

Kyiv – 2025

1. Problem Statement

- Reuse the code from practical assignment #2.
- Implement sound source rotation around the geometrical center of the surface patch by means of tangible interface (the surface stays still this time and the sound source moves). Reproduce your favorite song in mp3/ogg format having the spatial position of the sound source controlled by the user.
- Visualize position of the sound source with a sphere.
- Add a sound filter (use BiquadFilterNode interface) per variant. Add checkbox element that enables or disables the filter. Set filter parameters according to your taste.

2. Brief Theoretical Information

2.1 Spatial Audio Definition

Spatial audio, also known as 3D audio, refers to sound design and technology that mimics the natural way humans perceive sound in a three-dimensional space. It creates the illusion of sounds coming from different directions and distances, which provides a more immersive and realistic auditory experience. Spatial audio is often used in virtual reality (VR), gaming, and cinema to enhance the sense of presence and immersion in a digital environment.

2.2. Operating Principles of Spatial Audio

Spatial audio in VR adds to the immersion of an experience by simulating how people hear sounds in real life. Here's a breakdown of how it functions:

– *3D Sound Simulation*: Spatial audio replicates how sound behaves in a three-dimensional space. This means sounds in VR can seem to come from above, below, behind, in front of, or beside the listener. It's different from traditional stereo audio, which can only convey left and right directions.

– *Head-Related Transfer Function (HRTF)*: One of the key technologies behind spatial audio is the HRTF. It's a set of algorithms that mimic how our ears receive sounds from different directions and distances. HRTFs account for the fact that sounds are altered by the shape of our ears, head, and torso before reaching our eardrums. This alteration helps our brain understand the sound's location.

– *Binaural Recording Techniques*: To create spatial audio content, binaural recording is often used. This involves placing two microphones in the ears of a dummy head or a human to record sound exactly as it would be heard by a person. When played back through headphones, this creates a highly realistic and spatially accurate sound experience.

– *Directional Audio Cues*: In VR, as the user moves their head or navigates through the virtual environment, the spatial audio adjusts in real-time. If a sound source is to the right of the user and they turn their head towards it, the sound will naturally seem to come from straight ahead, just as it would in real life.

– *Distance and Environment Effects*: Spatial audio also simulates how sound changes with distance and in different environments. Sounds can become softer, more muffled, or change in quality as they get further away or as the environment changes (like echoing in a large hall or being absorbed in a densely furnished room).

– *Integration with VR Systems*: Spatial audio is tightly integrated with software and hardware (like a VR headset). The VR system tracks the user’s head movements and adjusts the audio in real-time to maintain the directionality and realism of sounds, enhancing the overall sense of presence in the virtual world.

2.3. HTML5 Web Audio API

The HTML5 Web Audio API is a powerful, high-level JavaScript framework for creating, processing, and spatializing audio directly in the browser. At its core lies the AudioContext, which serves both as the global timing engine and the container for an audio-processing graph composed of connected AudioNode objects. Developers assemble

complex audio pipelines by linking source nodes, processor nodes, spatialization nodes, and the final destination.

Key components include:

– *AudioContext*: The primary interface for all audio operations. It manages animation-frame-accurate scheduling and sample-level timing, ensuring synchronized playback and processing.

– *AudioBufferSourceNode*: A source node that plays back decoded audio data (AudioBuffer), supporting looping, playback rate adjustment, and precise start/stop control.

– *BiquadFilterNode*: A versatile filter node implementing second-order infinite impulse response (IIR) filters. It offers multiple filter types – low-pass, high-pass, band-pass, notch, shelving – and exposes frequency, Q, and gain parameters to shape the spectral content of the signal.

– *AnalyserNode*: A non-destructive node that provides real-time frequency and time-domain analysis. Developers can configure its fast Fourier transform (fftSize) and retrieve data arrays via getByteFrequencyData() or getByteTimeDomainData() for visualizations or interactive feedback.

– *PannerNode*: A 3D spatialization node that positions an audio source in three-dimensional space using models such as HRTF or equal-power. It exposes properties for positionX/Y/Z, orientationX/Y/Z, distance attenuation (refDistance, maxDistance, rolloffFactor), and the choice of distanceModel.

3. Implementation Details

3.1. Global State and Module Imports

At the top of the main.js file, a set of global variables was declared to support audio processing and its visual representation. These include the audioContext instance and the core audio nodes: sourceNode, filterNode, analyserNode, and pannerNode. A typed array frequencyData holds real-time amplitude values retrieved from the analyser node.

Additionally, a global audioSphereModel reference is used to render a 3D sphere that visually represents the audio source. The logic for generating and drawing this object is encapsulated in a separate module (audio-sphere-model.js), which defines a class that builds the sphere geometry based on configurable resolution parameters. It buffers the vertex and index data and exposes a draw() method, which is called during rendering. The implementation is provided in Appendix 5.1.

3.2. Audio File Loading and Decoding

To allow users to select and play custom audio files, a file input element was added:

```
<div class="mb-2">
  <label for="audioFile" class="form-label">Select Audio File: </label>
  <input type="file" class="form-control" id="audioFile" accept=".mp3, .ogg"
lang="en">
</div>
```

When a file is selected, the following event listener is triggered:

```
document.getElementById('audioFile').addEventListener('change', function () {
  const file = this.files[0];
  if (file) initAudioFromFile(file).catch(console.error);
});
```

The initAudioFromFile() function creates and resumes an AudioContext, decodes the file into an AudioBuffer, and creates a looping AudioBufferSourceNode. A BiquadFilterNode of type "band-pass" is also initialized with default settings, along with an AnalyserNode (FFT size 64) and a PannerNode configured for HRTF spatialization. Once all nodes are set up, the updateAudioGraph() function is called to wire them together, and playback begins via sourceNode.start(0). The full initAudioFromFile function code is listed in Appendix 5.2.

3.3. Dynamic Audio Graph & Real-Time Filter Controls

A checkbox input was added to allow users to toggle the band-pass filter dynamically:

```
<div class="mb-2">
  <label>
    <input type="checkbox" id="filterToggle"> Band-pass filter
  </label>
</div>
```

An event listener was attached to this input:

```
document.getElementById('filterToggle').addEventListener('change', function () {
  updateAudioGraph();
});
```

The updateAudioGraph() function resets and rebuilds the connections between sourceNode, filterNode, analyserNode, and pannerNode whenever the filter checkbox changes. It begins by disconnecting all existing links, then reconnects the nodes so that the audio either flows through the band-pass filter or bypasses it, feeds into the analyser, and finally into the panner before reaching the audio context's destination. The implementation is provided in Appendix 5.3.

To give users finer, real-time control, two range sliders were added alongside the checkbox. These sliders adjust the filter's frequency and Q properties directly – without rebuilding the audio graph – providing smooth, immediate auditory feedback as the user tweaks the band-pass settings.

3.4. Visual and Spatial Audio Synchronization

The draw() function was modified to visualize audio characteristics and control sound spatialization. Frequency amplitude is analyzed in real time using the AnalyserNode, and the average value is mapped to the scale of a 3D sphere. The virtual position of the sphere is determined by applying a 4×4 transformation matrix derived from orientation sensor data. This position is also assigned to the PannerNode, synchronizing the perceived direction of the audio with the rendered object in 3D space. The full rendering logic is provided in Appendix 5.4.

4. User's Instruction with Screenshots

When the page loads, the user sees the 3D canvas on the left, showing the corrugated spheres. If camera access is granted, the live webcam feed appears as the canvas background. On the right is the settings panel, which includes:

– numeric inputs for Convergence, Eye Separation, Field of View (FOV), Near Clipping Distance, and Far Clipping Distance;

– select Audio File button;

– Band-pass filter checkbox;

– Frequency and Quality sliders for real-time audio filter tuning.

The user can rotate the view by clicking and dragging with the mouse on desktop, or by physically rotating their phone (orientation data is sent via WebSocket).

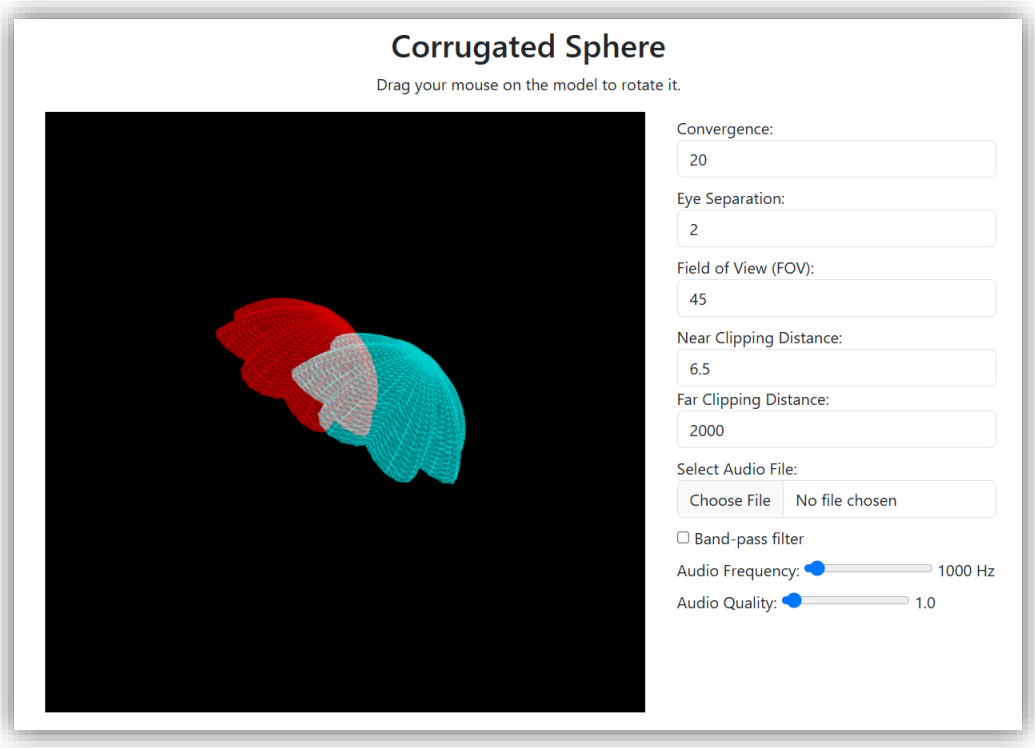

Figure 1 – The initial view of the 3D model and the settings panel on the right.

After clicking Select Audio File and choosing an MP3 or OGG file, audio playback begins immediately and a wireframe audio sphere appears at the center of the scene. This sphere pulses in size to the rhythm of the music but remains fixed at the model's center.

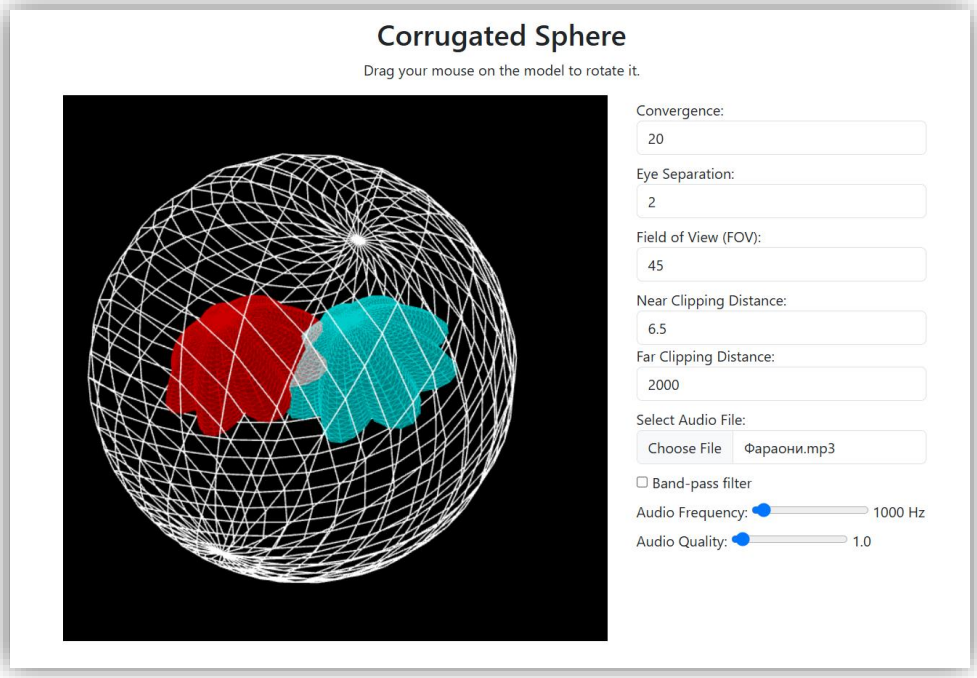

Figure 2 – Audio file loaded; the sphere pulses in response to the full-spectrum audio.

When the Band-pass filter checkbox is enabled, a band-pass filter is applied to the stream.

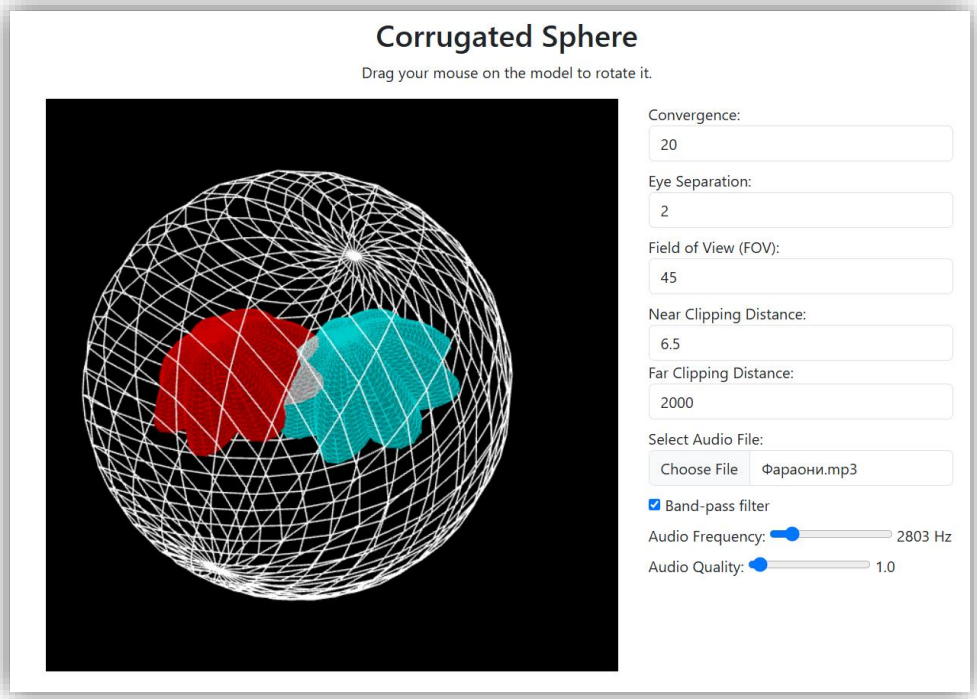

Figure 3 – Band-pass filter enabled; sphere pulses according to the filtered band.

5. Source Code

5.1. File “audio-sphere-model.js”

```
'use strict';

/**
 * Creates a simple sphere model.
 * @param {WebGLRenderingContext} gl - The WebGL rendering context.
 * @param {ShaderProgram} shaderProgram - The shader program to be used for
rendering the model.
 * @param {number} radius - The radius of the sphere.
 * @param {number} latBands - The number of latitude bands.
 * @param {number} longBands - The number of longitude bands.
 * @constructor
 */
export class SphereModel {
    constructor(gl, shaderProgram, radius, latBands, longBands) {
        this.gl = gl;
        this.shaderProgram = shaderProgram;
        this.radius = radius;
        this.latBands = latBands;
        this.longBands = longBands;
        this.vertexBuffer = this.gl.createBuffer();
        this.indexBuffer = this.gl.createBuffer();
        this.bufferData();
    }

    /** Generates vertices for drawing the sphere. **/
    getVertices() {
        const vertices = [];

        for (let lat = 0; lat <= this.latBands; lat++) {
            const theta = lat * Math.PI / this.latBands;
            const sinTheta = Math.sin(theta);
            const cosTheta = Math.cos(theta);

            for (let lon = 0; lon <= this.longBands; lon++) {
                const phi = lon * 2 * Math.PI / this.longBands;
                const sinPhi = Math.sin(phi);
                const cosPhi = Math.cos(phi);
                const x = this.radius * cosPhi * sinTheta;
                const y = this.radius * cosTheta;
                const z = this.radius * sinPhi * sinTheta;

                vertices.push(x, y, z);
            }
        }

        return vertices;
    }

    /** Generates indices for drawing the sphere. **/
```

```
    getIndices() {
        const indices = [];

        for (let lat = 0; lat < this.latBands; lat++) {
            for (let lon = 0; lon < this.longBands; lon++) {
                const first = lat * (this.longBands + 1) + lon;
                const second = first + this.longBands + 1;

                indices.push(first, second, first + 1);
                indices.push(second, second + 1, first + 1);
            }
        }

        return indices;
    }

    /** Buffers the vertex and index data in the WebGL context. **/
    bufferData() {
        const vertices = this.getVertices();
        this.gl.bindBuffer(this.gl.ARRAY_BUFFER, this.vertexBuffer);
        this.gl.bufferData(this.gl.ARRAY_BUFFER, new Float32Array(vertices),
this.gl.STATIC_DRAW);

        const indices = this.getIndices();
        this.gl.bindBuffer(this.gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);
        this.gl.bufferData(this.gl.ELEMENT_ARRAY_BUFFER, new Uint16Array(indices),
this.gl.STATIC_DRAW);
    }

    /** Draws the sphere surface using buffered vertex and index data. **/
    draw() {
        this.gl.bindBuffer(this.gl.ARRAY_BUFFER, this.vertexBuffer);
        this.gl.vertexAttribPointer(this.shaderProgram.aVertex, 3, this.gl.FLOAT,
false, 0, 0);
        this.gl.enableVertexAttribArray(this.shaderProgram.aVertex);

        this.gl.uniform3fv(this.shaderProgram.uColor, new Float32Array([1.0, 1.0,
1.0]));
        this.gl.bindBuffer(this.gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);
        this.gl.drawElements(this.gl.LINES, this.latBands * this.longBands * 6,
this.gl.UNSIGNED_SHORT, 0);
    }
}
```

5.2. Function “initAudioFromFile(file)”

```
async function initAudioFromFile(file) {
    audioContext = new (window.AudioContext || window.webkitAudioContext)();
    await audioContext.resume();

    const arrayBuffer = await file.arrayBuffer();
    audioContext.decodeAudioData(arrayBuffer, (decodedBuffer) => {
```

```
        sourceNode = audioContext.createBufferSource();
        sourceNode.buffer = decodedBuffer;
        sourceNode.loop = true;

        filterNode = audioContext.createBiquadFilter();
        filterNode.type = 'bandpass';
        filterNode.frequency.value = 1000;
        filterNode.Q.value = 1;

        analyserNode = audioContext.createAnalyser();
        analyserNode.fftSize = 64;
        frequencyData = new Uint8Array(analyserNode.frequencyBinCount);

        pannerNode = audioContext.createPanner();
        pannerNode.panningModel = 'HRTF';
        pannerNode.distanceModel = 'inverse';
        pannerNode.positionX.value = 0;
        pannerNode.positionY.value = 0;
        pannerNode.positionZ.value = 0;

        updateAudioGraph();
        sourceNode.start(0);
    }, (error) => console.error('Error decoding audio data:', error));
}
```

5.3. Function “updateAudioGraph()”

```
function updateAudioGraph() {
    sourceNode.disconnect();
    filterNode.disconnect();
    analyserNode.disconnect();
    pannerNode.disconnect();

    const filterOn = document.getElementById('filterToggle').checked;
    if (filterOn) {
        sourceNode.connect(filterNode);
        filterNode.connect(analyserNode);
    } else {
        sourceNode.connect(analyserNode);
    }

    analyserNode.connect(pannerNode);
    pannerNode.connect(audioContext.destination);
}
```

5.4. Function “draw()” (file “main.js”)

```
function draw() {
    gl.clearColor(0, 0, 0, 1);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);

    gl.disable(gl.DEPTH_TEST);
    renderVideoBackground();
    gl.enable(gl.DEPTH_TEST);

    let translate = m4.translation(0, 0, -10);
    let modelView = spaceBall.getViewMatrix();

    if (!sensorRotationMatrix4) {
        sensorRotationMatrix4 = m4.identity();
    }
    modelView = m4.multiply(sensorRotationMatrix4, modelView);
    modelView = m4.multiply(translate, modelView);

    let leftProjection = m4.identity();
    let rightProjection = m4.identity();
    let modelViewProjection;

    updateStereoCamera();
    shaderProgram.use();
    gl.uniform3fv(shaderProgram.uColor, [1.0, 1.0, 1.0]);

    gl.colorMask(true, false, false, true);
    stereoCamera.applyFrustum(modelView, leftProjection, 'left');
    modelViewProjection = m4.multiply(leftProjection, modelView);
    gl.uniformMatrix4fv(shaderProgram.uModelViewProjectionMatrix, false,
modelViewProjection);
```

```
    if (!model) {
        model = new Model(gl, shaderProgram, 1, 0.4, 4, 50, 50);
    }
    model.draw();
    model.drawWireframe();

    gl.colorMask(false, true, true, true);
    stereoCamera.applyFrustum(modelView, rightProjection, 'right');
    modelViewProjection = m4.multiply(rightProjection, modelView);
    gl.uniformMatrix4fv(shaderProgram.uModelViewProjectionMatrix, false,
modelViewProjection);
    model.draw();
    model.drawWireframe();

    gl.colorMask(true, true, true, true);

    if (analyserNode && pannerNode) {
        analyserNode.getByteFrequencyData(frequencyData);

        const averageAmplitude = frequencyData.reduce((sum, value) => sum + value,
0) / frequencyData.length;
        const sphereScale = 1 + (averageAmplitude / 255) * 0.5;
        const rotationMatrix = sensorRotationMatrix4 || m4.identity();
        const [posX, posY, posZ] = m4.transformPoint(rotationMatrix, [0, 0, 0, 1]);

        pannerNode.positionX.value = posX;
        pannerNode.positionY.value = posY;
        pannerNode.positionZ.value = posZ;

        const sphereTranslationMatrix = m4.translation(posX, posY, posZ);
        const sphereScaleMatrix = m4.scaling(sphereScale, sphereScale,
sphereScale);
        const sphereModelMatrix = m4.multiply(sphereTranslationMatrix,
sphereScaleMatrix);
        const sphereModelViewProjection = m4.multiply(leftProjection,
m4.multiply(modelView, sphereModelMatrix));

        gl.uniformMatrix4fv(shaderProgram.uModelViewProjectionMatrix, false,
sphereModelViewProjection);
        if (!audioSphereModel) {
            audioSphereModel = new SphereModel(gl, shaderProgram, 2.5, 24, 24);
        }
        audioSphereModel.draw();
        gl.uniform3fv(shaderProgram.uColor, [1.0, 1.0, 1.0]);
    }
}
```